ET5 IIM
Module: Machine Learning II
Teacher: Nasredine SEMMAR

Project

You can use a shared Git repository among your group members to ensure compatibility between the changes you make. This Git repository should be structured as follows:

- A text file named **README** containing the instructions needed to run the Python scripts you have developed. This should include command-line options and usage examples.
- A "**src**" folder for the source code of your Python scripts. Code readability will be one of the evaluation criteria (precise, concise, clear, and well-spelled comments).
- A document containing your report, written in LaTeX or Word and compiled into a 5- to 7-page PDF named name1-name2-name3-name4.pdf, describing your work. In it, you will describe the project's objective, the evaluation results of the different neural models, their strengths and limitations. If you know how to overcome these limitations, mention it and describe your ideas, presenting them as avenues for future work. You must also include a short section describing each team member's contribution. The report can be written in English or French.
- A **data** folder for the manipulated data.

You will send me by email (nasredine.semmar@cea.fr) only a GitHub link to your project no later than Tuesday, March 10, 2026.

Notes:

- If you don't want to use GitHub, you can send me the project report and the notebook.
- You will have to choose one of the two subjects.

<u>Subject 1:</u> Transfer Learning through feature extraction from a CNN

Goals

The goal of this project is to become acquainted with a well-known and commonly-used deep CNN network: *VGG16* (Simonyan and Zisserman, 2015). Then, the method described by Chatfield et al. (2014) will be used with this network on the *15-Scene* dataset. We will particularly look at the strategy to extract *deep* features thanks to a pre-trained network, and we will use these features in a classic classification scheme using linear SVM.

Note: For the implementation of the neural models, you follow the instructions in the provided notebook (Transfer_Learning.ipynb).

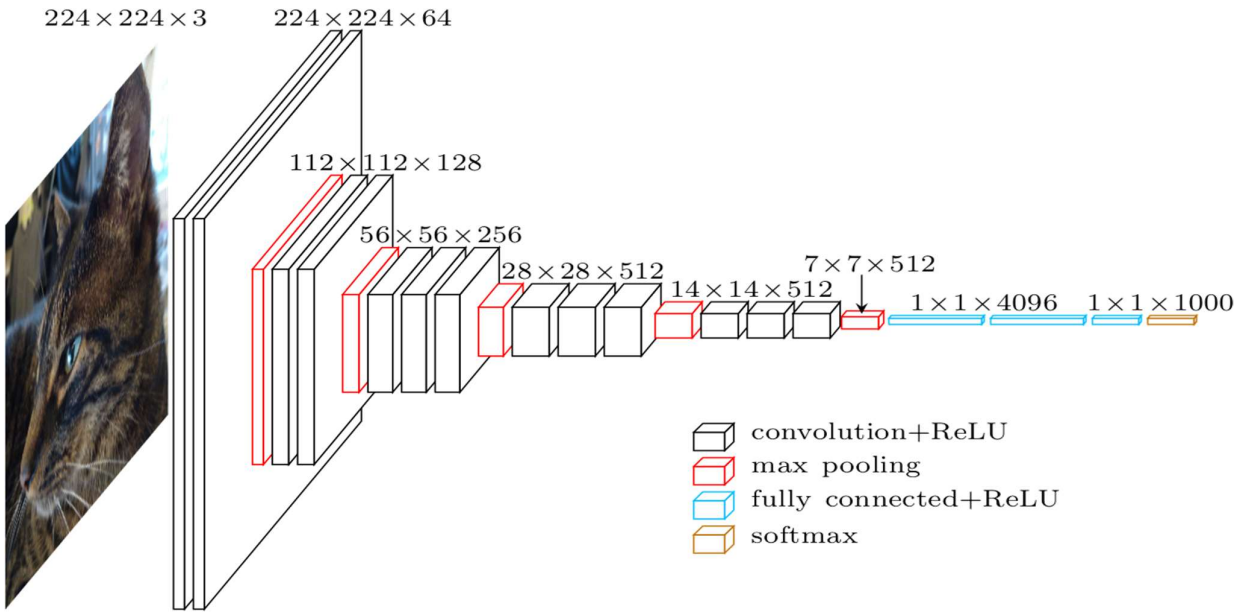

Figure 1: VGG16 Network

Section 1 – VGG16 Architecture

The convolutional network VGG16 (Figure 1) is a classification model trained on the ImageNet dataset (Russakovsky et al., 2015). This dataset contains over 1 million images categorized in 1000 classes. Its architecture consists in five blocks, each composed of:

- Two or three convolutional layers preserving the spatial dimensions.
- A pooling layer which halves the spatial dimensions.

Then, the number of feature maps are doubled at each following block. The end of the network contains three fully-connected layers.

The pre-trained VGG16 network can be loaded in PyTorch with the following command:

```
import torchvision
vgg16 = torchvision.models.vgg16(pretrained=True)
```

The images given as input in the network need to be resized to 224×224 pixels, and normalized with the mean $\mu = [0.485, 0.456, 0.406]$ and the standard deviation $\sigma = [0.229, 0.224, 0.225]$ (calculated on the training set of ImageNet). The list with the 1000 classes from ImageNet can be obtained with a provided dictionary. Loading the dictionary and an image can be achieved with the following code:

```python
import pickle
from PIL import Image
import numpy as np

nameim = "cat"+".jpg"
img = Image.open(nameim)
plt.imshow(img)

# Loading ImageNet classes
imagenet_classes = pickle.load(open('imagenet_classes.pkl', 'rb'))

# Normalization
img = img.resize((224, 224), Image.BILINEAR)
img = np.array(img, dtype=np.float32) / 255
img = img.transpose((2, 0, 1))
# ImageNet mean/std
mu = torch.Tensor([0.485, 0.456, 0.406])
sigma = torch.Tensor([0.229, 0.224, 0.225])
# Expand mu & sigma to match image size + compute normalized image
# YOUR CODE HERE

# Loding pre-trained VGG
vgg16 = torchvision.models.vgg16(pretrained=True)
vgg16.eval() # WHY THAT ?

# Forward pass on VGG
img = np.expand_dims(img, 0)
x = torch.Tensor(img)
y = ... # TODO calcul forward
y = y.numpy() # transformation en array numpy

# Get prediction (i.e., ImageNet class label)
# YOUR CODE HERE
```

Questions

1. Knowing that the fully-connected layers account for the majority of the parameters in a model, give an estimate on the number of parameters of VGG16 (using the sizes given in Figure 1).
2. What is the output size of the last layer of VGG16? What does it correspond to?
3. Apply the network on several images of your choice and comment on the results.
   - What is the role of the ImageNet normalization?
4. Visualize several activation maps obtained after the first convolutional layer. How can we interpret them?

Section 2 - Transfer Learning with VGG16 on 15-Scene dataset

2.1 Approach

General Principle The principle of the approach is rather simple and consists in two steps: we will first use a pre-trained neural network to perform *feature extraction* which we then use to train a classification network.

The first step can be seen as an alternative on the Bag-of-words (BoW) approach with SIFT (Scale-Invariant Feature Transform) descriptors (Lowe, 2004). The goal is to obtain a vector representation for every image which we can then later use for a variety of tasks, particularly image classification.

The principle of *feature extraction* consists in producing this image representation using a network which was trained to solve a different task than ours. In our case, the VGG16 network was trained for the task of image classification on the ImageNet dataset. To produce the image representations, we will use the output of intermediate layers of the chosen pre-trained network.

In the case of this project, we will use the VGG16 network, and we will represent each image as the vector outputted from the relu7 layer (the ReLU just before the classification layer). We will then train an SVM classifier on each class of the *15-Scene* dataset to solve our classification task.

Questions

1. Why not directly train VGG16 on 15-Scene? dataset
2. How can pre-training on ImageNet help classification for 15-Scene dataset?
3. What limits can you see with feature extraction?

2.2 Feature Extraction with VGG16

To extract the features at the *relu7* layer of VGG16, create a new class VGG16relu7 where you will copy the VGG16 layers up until the *relu7* (see the code below). The output of this new network will be the desired features. To extract the features from another layer, you can modify the classifier field (or potentially features) to delete more layers.

The input images to the network need to be resized to the correct shape (with the correct number of channels) and normalized with the mean and standard deviation given above.

Using this procedure, create the matrices $\mathbf{X}_{train}$ and $\mathbf{X}_{test}$ in which each row corresponds to a feature vector. Each feature vector will likewise be normalized with the L2 norm.

Questions

1. What is the impact of the layer at which the features are extracted?
2. The images from 15-Scene dataset are black and white, but VGG16 requires RGB images. How can we get around this problem?

```
class VGG16relu7(nn.Module):
    def __init__(self):
        super(VGG16relu7, self).__init__()
        # Copy the entire convolutional part
        self.features = nn.Sequential( *list(vgg16.features.children()))
        # Keep a piece of the classifier: -2 to stop at relu7
        self.classifier = nn.Sequential(*list(vgg16.classifier.children())[:-2])

    def forward(self, x):
        x = self.features(x)
        x = x.view(x.size(0), -1)
        x = self.classifier(x)
        return x
```

2.3 Training SVM classifiers

Using the *features* ($\mathbf{X}_{train}$, $\mathbf{X}_{test}$) and the corresponding *targets* (y_{train}, y_{test}), we will train a multi-class classifier (composed of many binary linear SVM classifiers in *one-versus-all*) with these elements. You will use sklearn.svm.LinearSVC from Scikit Learn.

```
from sklearn.svm import LinearSVC
svm = LinearSVC(C=1.0)
```

Train a multi-class SVM classifier on the training set with $C = 1$ and evaluate its classification score (*accuracy*) on the test set using the *deep features* previously extracted. For this, the functions `fit` and score from LinearSVC will be used to train the SVM and calculate its performance.

Question

1. Rather than training an independent classifier, is it possible to just use the neural network? Explain.

2.4 Going further

Once this classification scheme works, you can study the effect of two (or more) choices on the performances or training time. Some possible examples are listed below but you are free to find others.

- Change the layer at which the *features* are extracted. What is the importance of the depth of this layer? What is the representation size and what does this change?
- Try other available pre-trained networks. What are the differences between these networks?
- Tune the parameter C to improve performance.
- Instead of training an SVM, replace the last layer of VGG16 with a new fully-connected layer and continue to train the network on 15 Scene (with or without propagating the gradients to the rest of the network).
- Look into methods for dimensionality reduction before classification and their impact on performance and execution time.

Question

1. For every improvement that you test, explain your reasoning and comment on the obtained results.

References

- Ken Chatfield, Karen Simonyan, Andrea Vedaldi, and Andrew Zisserman. Return of the devil in the details: Delving deep into convolutional nets. In *Proceedings of the British Machine Vision Conference (BMVC)*, 2014.
- Olga Russakovsky, Jia Deng, Hao Su, Jonathan Krause, Sanjeev Satheesh, Sean Ma, Zhiheng Huang, Andrej Karpathy, Aditya Khosla, Michael Bernstein, Alexander C. Berg, and Li Fei-Fei. Imagenet large scale visual recognition challenge. *Internationale Journal of Computer Vision (IJCV)*, 2015.
- Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2015.
- David G. Lowe, Distinctive image features from scale-invariant keypoints, International Journal of Computer Vision, vol. 60, no 2, 2004: http://www.cs.ubc.ca/~lowe/papers/ijcv04.pdf, https://blog.roboflow.com/sift/.

<u>Subject 2:</u> Explainable and Trustworthy

Section 1 - Multi-choice questions

What does local interpretability in Explainable Artificial Intelligence (XAI) refer to?

A) Understanding the overall behavior of the model

B) Understanding individual predictions or decisions made by the model

C) Understanding the data preprocessing steps in the model

D) Understanding the training process of the model

Which of the following best describes accountability in the context of Trustworthy AI?

A) The requirement that AI systems operate without any human oversight

B) The responsibility of the entities involved to ensure and answer for the outcomes produced by AI systems

C) The ability of AI systems to self-correct when errors occur

D) The focus on minimizing the cost of deploying AI systems

Section 2 - Open questions – Short answers

Discuss how counterfactual explanations can be used to provide insights into model predictions. Provide an example scenario where counterfactual explanations would be useful.

Section 3 - Open questions – Long answers

Consider a trained black box healthcare AI system operating with tabular data that predicts the likelihood of disease.

- How would you ensure that this system provides understandable explanations to doctors?
- Discuss some methods and the types of explanations that would be appropriate.

Section 4 – Implementation of Explainable AI Examples

Use the diabetes dataset from Kaggle to illustrate the functioning of LIME and SHAP tools.

Note: You can refer to the following blog for your inspiration:
https://www.datacamp.com/tutorial/explainable-ai-understanding-and-trusting-machine-learning-models?utm_cid=19589720821&utm_aid=157156374951&utm_campaign=230119_1-ps-other~dsa-tofu~all_2-b2c_3-emea_4-prc_5-na_6-na_7-le_8-pdsh-go_9-nb-e_10-na_11-na&utm_loc=1006111-&utm_mtd=-c&utm_kw=&utm_source=google&utm_medium=paid_search&utm_content=ps-other~emea-en~dsa~tofu~tutorial~machine-learning&gad_source=1&gad_campaignid=19589720821&gbraid=0AAAAADQ9WsFR5FcpfkAwKRf3fOdzCKvW7&gclid=EAIaIQobChMIhva-9feDkgMVqDIGAB2ZQDi4EAAYBCAAEgLOu_D_BwE).